

DA5300: Data Structures For Data Science

Lab Exam I

23 September 2026

Maximum Marks: 15

Problem Statement

Implement a **Binary Search Tree (BST)** that supports insertion, deletion, structural queries, and tree traversals. A Binary Search Tree satisfies the property that every key in the left subtree is smaller than the node key and every key in the right subtree is greater than the node key.

Required Operations

Implement the following seven operations in the `BST` class.

- (a) **Insertion:** Insert a key into the BST.
- (b) **Deletion:** Delete a key from the BST.
- (c) **Height:** Return the height of the BST.
- (d) **Number of Nodes:** Return the total number of nodes.
- (e) **Inorder Traversal:** Return the inorder traversal.
- (f) **Preorder Traversal:** Return the preorder traversal.
- (g) **Postorder Traversal:** Return the postorder traversal.

Important Conventions

1. The height of an **empty tree** is -1 .
2. The height of a tree containing only the root is 0 .
3. Height is measured as the number of **edges** on the longest root-to-leaf path.
4. Duplicate insertions must be **ignored**.
5. Deleting a key that is not present must leave the tree unchanged.
6. For deletion of a node having two children, use the inorder successor.

Input Format

The program will be executed as:

```
python bst.py test.txt
```

The input file contains one command per line.

Command	Meaning
I x	Insert key x
D x	Delete key x
H	Print height of the tree
N	Print number of nodes
IN	Print inorder traversal
PRE	Print preorder traversal
POST	Print postorder traversal

For example:

```
I 50
I 30
I 70
I 20
I 40
I 60
I 80
H
N
IN
PRE
POST
```

produces:

```
2
7
20 30 40 50 60 70 80
50 30 20 40 70 60 80
20 40 30 60 80 70 50
```

Program Structure

A starter file `bst.boilerplate.py` will be provided. Complete the missing implementations in the BST class. Retain the required command-line interface.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None
```

```
def insert(self, key):  
    # implement  
  
def delete(self, key):  
    # implement  
  
def height(self):  
    # implement  
  
def number_of_nodes(self):  
    # implement  
  
def inorder(self):  
    # implement  
  
def preorder(self):  
    # implement  
  
def postorder(self):  
    # implement
```

Deletion

Deletion must correctly handle: (1) a leaf node, (2) a node with exactly one child, and (3) a node with two children. For two children, you may use either the minimum key in the right subtree (inorder successor) or the maximum key in the left subtree (inorder predecessor), provided the resulting BST is correct.

Output Format

For every query command, print exactly one corresponding line. H and N print one integer. IN, PRE, and POST print keys separated by spaces. Insertion and deletion commands produce no output.

Restrictions

1. Do not use Python's built-in sorting functions to implement BST operations.
2. Do not use external BST/tree libraries.
3. The tree must actually be maintained as a binary search tree.
4. Traversals must traverse the tree rather than sorting a separate collection of keys.
5. Recursion or iteration may be used.

Testing and Marks

Test Category	Number of Tests	Marks
Public tests	5	5
Private tests	10	10
Total	15	15

Each public test is worth 1 mark. You may run the public evaluator to check your current score. The private tests are used by the examiner for final evaluation. A test is passed only when all required outputs for that test are correct.

Student Testing

The five public test cases are provided separately.

First complete the program in `bst_boilerplate.py`. For testing during the examination, save your completed program as:

```
bst.py
```

Then run:

```
python public_evaluate.py
```

The evaluator will report your current score out of 5.

Important: `bst.py` is only the filename used for development and public testing. You do **not** submit a separate file called `bst.py`.

Submission

At the end of the examination, submit the **same completed program**, but rename the file using your roll number.

For example, if your roll number is DA26ABCD, rename:

```
bst.py
```

to:

```
DA26ABCD.py
```

There is **no difference in the program itself** between `bst.py` and `23CS1001.py`; only the filename changes.

The complete workflow is:

```
bst_boilerplate.py
|
|   complete the TODOs
v
bst.py
|
|   run public tests
```

```
      v
public_evaluate.py
      |
      |  after testing
      v
DA26ABCD.py
      |
      |  submit
      v
Examiner
```

The final submitted program must accept the input test file as its first command-line argument. For example:

```
python DA26ABCD.py test.txt
```

The examiner will run the submitted roll-number file automatically on the public and private test cases.

Do not include additional output such as debugging messages, prompts, explanations, or labels.

Submit only the roll-number-named Python file.

Good luck!